Apresentação dos testes

Nodify — Sistema de Acessibilidade Cefálica para Spotify

1. O que você precisa

Notebook com o projeto aberto	`C:\Users\pedro\acessibilidade-spotify`
Terminal	Git Bash ou PowerShell, na pasta do projeto
Internet	**Não** — as dependências já estão em cache
Celular / emulador	**Não** — os testes rodam na JVM
Spotify aberto	**Não**

Só o notebook. Nada mais.

2. Comandos

Antes de apresentar — faça em casa, uma vez (~1 minuto, porque compila)

```
cd C:\Users\pedro\acessibilidade-spotify
./gradlew testDebugUnitTest -Pdemo
```

Na frente do professor — o mesmo comando (~11 segundos)

```
./gradlew testDebugUnitTest -Pdemo

GestureClassifierTest > detecta TILT_HEAD_RIGHT ... PASSED
PlayerAtivoViewModelTest > aceno dispara tocar ..... PASSED
                                        (… 61 linhas)
BUILD SUCCESSFUL
```

Se pedirem a suíte completa

```
./gradlew test          # 186 testes, ~4 segundos
```

Se pedirem a cobertura

```
./gradlew koverHtmlReportDebug
start app\build\reports\kover\htmlDebug\index.html
```

3. O que está sendo testado

61 testes, 4 classes — uma por etapa do produto:

#	Etapa	Arquivo de teste	Arquivo testado
1	Detectar o gesto	`GestureClassifierTest.kt`	`vision/GestureClassifier.kt`
2	Calibrar o usuário	`CalibrationThresholdCalculatorTest.kt`	`domain/gesture/CalibrationThresholdCalculator.kt`
3	Customizar mapeamento	`GestureConfigViewModelTest.kt`	`ui/screens/GestureConfigViewModel.kt`
4	**Gesto vira comando**	`PlayerAtivoViewModelTest.kt`	`ui/screens/PlayerAtivoViewModel.kt`

Testes em `app/src/test/java/com/sac/acessibilidade/…` · Código em `app/src/main/java/com/sac/acessibilidade/…`

4. O código para mostrar

Abra `app/src/main/java/com/sac/acessibilidade/ui/screens/PlayerAtivoViewModel.kt`, **linha 57**:

```
private fun collectGestures() {
    viewModelScope.launch {
        gestureProcessor.gestureFlow.collect { gesture ->
            val action = currentMapping[gesture] ?: return@collect
            showGestureFeedback(gesture, action)
            executeAction(action)
        }
    }
}
```

Aponte a linha destacada e diga:

> "A ação vem do mapa carregado do banco, não do código. Se o gesto não tem ação, sai em silêncio. É isso que permite o usuário usar só os gestos que ele consegue fazer."

Os testes que provam essa linha aparecem na tela quando você roda:

- `mapeamento customizado do usuario e respeitado`
- `remapeamento em tempo de execucao muda o comando disparado`
- `gesto sem acao mapeada e ignorado`

5. Perguntas prováveis

“Como testam sem celular?”

A camada de regra de negócio não importa nada de Android. O classificador recebe números e devolve um gesto; o ViewModel recebe os repositórios por construtor, substituídos por simulações no teste.

“Qual a cobertura?”

71,4 % do núcleo lógico, 22 % do app inteiro. A diferença são 1.300 linhas de Compose, que descrevem a tela e não têm lógica. Domínio, dados, integração com o Spotify e os 4 ViewModels estão em **100 %**.

“E a interface, é testada?”

Sim, de duas formas. Manualmente no aparelho (Galaxy S25+, Android 15): são os TF01–TF08 e os TI01–TI11 do documento, todos aprovados. E de forma automatizada, com 51 testes de Compose UI Test e 33 de integração — esses estão implementados e compilam, mas exigem celular conectado e ainda não foram executados.

“O documento fala em 186 testes, mas você rodou 61. Por quê?”

Os 186 são a suíte unitária completa. Os 61 são um recorte das quatro funcionalidades principais, para a demonstração caber no tempo. Rode `./gradlew test` que os 186 aparecem.

“Por que MockK e não Mockito?”

Classes Kotlin são `final` por padrão; o Mockito precisaria de configuração extra. O MockK também simula função `suspend` sem adaptação.

6. Não faça

> **Não rode `./gradlew connectedAndroidTest`** — precisa de celular conectado. Falha na hora.

> **Não rode pela primeira vez na frente dele** — compila e demora ~1 minuto.